SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION WITH OPENCV
COURSE CODE: DSA0204

COURSE OUTCOME COVERED

CO5: *Understand video processing - motion computation - 3D vision - geometry.* (BTL 6)

Assessment Tool Weightage: 40%

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)

A company wants to deploy a face recognition system on low-power edge devices (e.g., Raspberry Pi). The system must:

A. **Process live video streams in real time (<30 FPS latency)**
B. **Work under varying lighting conditions**
C. **Use limited memory and CPU resources**
D. **Detect and recognize faces from a predefined dataset**

Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

2. Automated Video Surveillance with Alert System

A security firm needs a video surveillance system that:

A. **Detects motion and unusual activities**
B. **Generates alerts in real time**
C. **Works with standard CCTV video feeds**
D. **Minimizes false positives in dynamic environments**

Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

3. OCR System for Low-Quality Documents

A digitization company needs an OCR system that:

A. **Extracts text from low-resolution, noisy scanned images**
B. **Handles varying fonts and illumination**
C. **Works efficiently for batch processing**

Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

ANSWERS

1. Real-Time Face Recognition System – Edge Deployment

Problem Analysis

The system must recognize faces from a live camera while running on a low-power device such as Raspberry Pi. Therefore, the pipeline should use lightweight preprocessing, fast face detection, and efficient feature extraction.

Proposed Pipeline

Input Camera → Preprocessing → Face Detection → Feature Extraction → Face Recognition → Output

Steps

1. **Image Acquisition**
    - Capture live video using an OpenCV camera.
    - Use cv2.VideoCapture() to read frames.
    - Resize frames to a smaller resolution such as 640 × 480 to reduce CPU usage**.**
2. **Preprocessing**
    - Convert the frame from BGR to grayscale using cv2.cvtColor().
    - Apply histogram equalization or CLAHE to improve images under different lighting conditions.
    - Resize the face region to a fixed size.
3. **Face Detection**

- Use a lightweight OpenCV Haar Cascade or LBPH-compatible face detector.
- Detect the location of faces using detectMultiScale().
- Process every few frames instead of detecting faces on every frame to save CPU.

4. **Feature Extraction**
    - Extract the detected face region.
    - Normalize the face image.
    - Use a compact face descriptor/embedding to represent facial features.
5. **Face Recognition**
    - Compare the extracted features with a predefined database.
    - Use LBPH (Local Binary Patterns Histograms) for a lightweight OpenCV-based implementation.
    - Calculate the similarity/confidence and assign the corresponding person's name.
6. **Output**
    - Draw a rectangle around the detected face.
    - Display the recognized person's name and confidence.
    - If no match is found, display "Unknown".

Constraint Satisfaction

Constraint	**Solution**
Real-time processing	Resize frames and use lightweight detection
Varying lighting	Grayscale + CLAHE/histogram equalization
Limited memory	Store only compact face features
Limited CPU	Process selected frames and use lightweight algorithms
Predefined dataset	Compare detected faces with stored face database

Result

The proposed OpenCV pipeline can provide fast face detection and recognition on low-power edge devices while reducing CPU and memory usage. Lighting normalization improves recognition accuracy in different environments.

2. Automated Video Surveillance with Alert System

Problem Analysis

The surveillance system must continuously monitor CCTV footage, identify moving objects or unusual activities, and generate alerts while avoiding false alarms caused by noise, trees, shadows, or other dynamic background objects**.**

Proposed Pipeline

CCTV Feed → Frame Acquisition → Preprocessing → Background Subtraction → Motion Detection → Feature Analysis → Activity Verification → Alert

Steps

1. **Video Acquisition**
    - Read the CCTV stream using OpenCV VideoCapture().
    - The input can be a camera feed, IP camera, or recorded CCTV video.
2. **Preprocessing**
    - Resize frames to reduce processing time.
    - Convert frames to grayscale.
    - Apply Gaussian blur using cv2.GaussianBlur() to remove small image noise.
3. **Background Subtraction**
    - Use OpenCV methods such as MOG2 or KNN.
    - Separate moving objects from the background**.**
4. **Motion Detection**
    - Generate a foreground mask.
    - Apply morphological operations such as opening and closing.
    - Find contours using cv2.findContours().

5. **Feature Detection**
    - Calculate object properties such as:
        - Bounding box
        - Area
        - Centroid
        - Direction of movement
        - Speed of movement
6. **Unusual Activity Analysis**
    - Define rules for suspicious activities.
    - Examples:
        - Person entering a restricted area
        - Object remaining in an area for a long time
        - Movement during restricted hours
        - Sudden abnormal movement
7. **False Positive Reduction**
    - Ignore very small contours caused by noise.
    - Use minimum object-area thresholds.
    - Require motion to continue for multiple frames before generating an alert.
    - Use morphological filtering to remove small unwanted regions**.**
8. **Alert Generation**
    - When suspicious activity is confirmed:
        - Draw a bounding box.
        - Save the frame/video.
        - Generate an alarm or notification.

Constraint Satisfaction

Constraint	Solution
Motion detection	MOG2/KNN background subtraction
Real-time alerts	Process frames continuously
CCTV compatibility	OpenCV VideoCapture()
Noise reduction	Gaussian blur + morphology
Dynamic background	Background subtraction with adaptive learning
False positives	Area threshold + multi-frame verification

Result

The system continuously monitors CCTV footage and identifies meaningful motion. Noise filtering, contour-area thresholds, and multi-frame verification help reduce false alarms while maintaining real-time performance**.**

3. OCR System for Low-Quality Documents

Problem Analysis

Low-quality scanned documents may contain blur, noise, uneven illumination, and different font sizes. Therefore, image enhancement and proper segmentation are required before OCR.

Proposed Pipeline

Input Document → Grayscale → Noise Removal → Illumination Correction → Thresholding → Morphological Processing → Segmentation → OCR → Text Output

Steps

1. **Image Acquisition**
    - Read scanned documents using OpenCV.
    - Use cv2.imread() to load document images.
2. **Grayscale Conversion**

- Convert the image to grayscale using: cv2.cvtColor()
- This reduces the amount of data and simplifies further processing.

3. **Noise Removal**
   - Apply Gaussian or median filtering.
   - Median filtering is useful for removing salt-and-pepper noise while preserving character edges**.**
4. **Contrast Enhancement**
   - Apply histogram equalization or CLAHE.
   - This improves the visibility of characters in poorly illuminated documents.
5. **Thresholding**
   - Convert the image into foreground and background.
   - Use adaptive thresholding for documents with uneven illumination.
   - OpenCV function: cv2.adaptiveThreshold().
6. **Morphological Processing**
   - Use erosion and dilation where required.
   - Opening can remove small noise.
   - Closing can connect broken parts of characters.
7. **Text Segmentation**
   - Detect text regions using contours or connected components.
   - Separate lines or individual text regions.
   - This helps OCR process the document more effectively**.**
8. **OCR Recognition**
   - Pass the cleaned and segmented image to an OCR engine.
   - The OCR engine identifies characters and converts them into machine-readable text.
9. **Post-Processing**
   - Remove unwanted characters.

- Correct common recognition errors.
- Combine recognized text from different regions**.**

10. Output

- Store the extracted text in a text file or database.
- For batch processing, process multiple document images automatically**.**

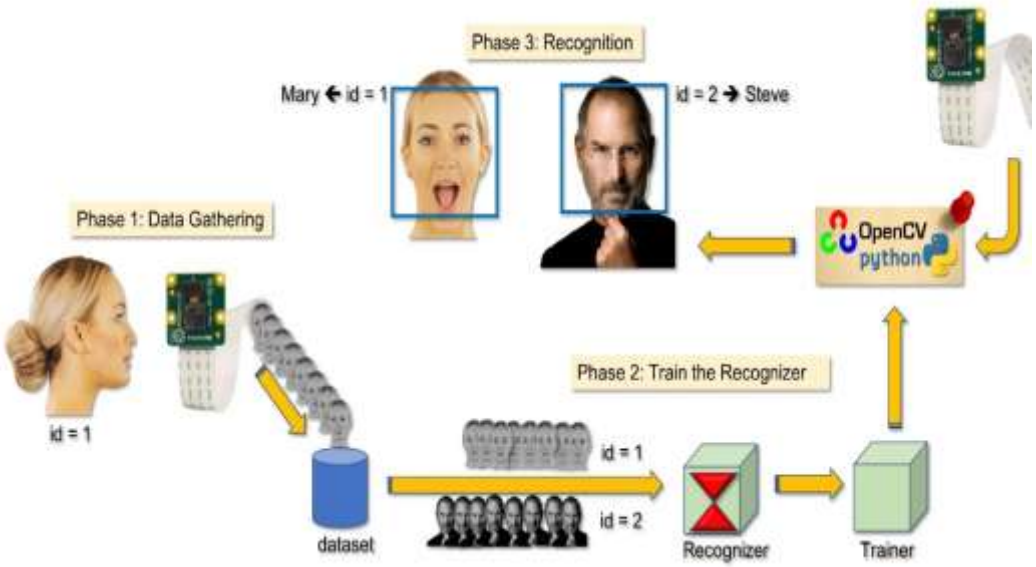
Constraint Satisfaction

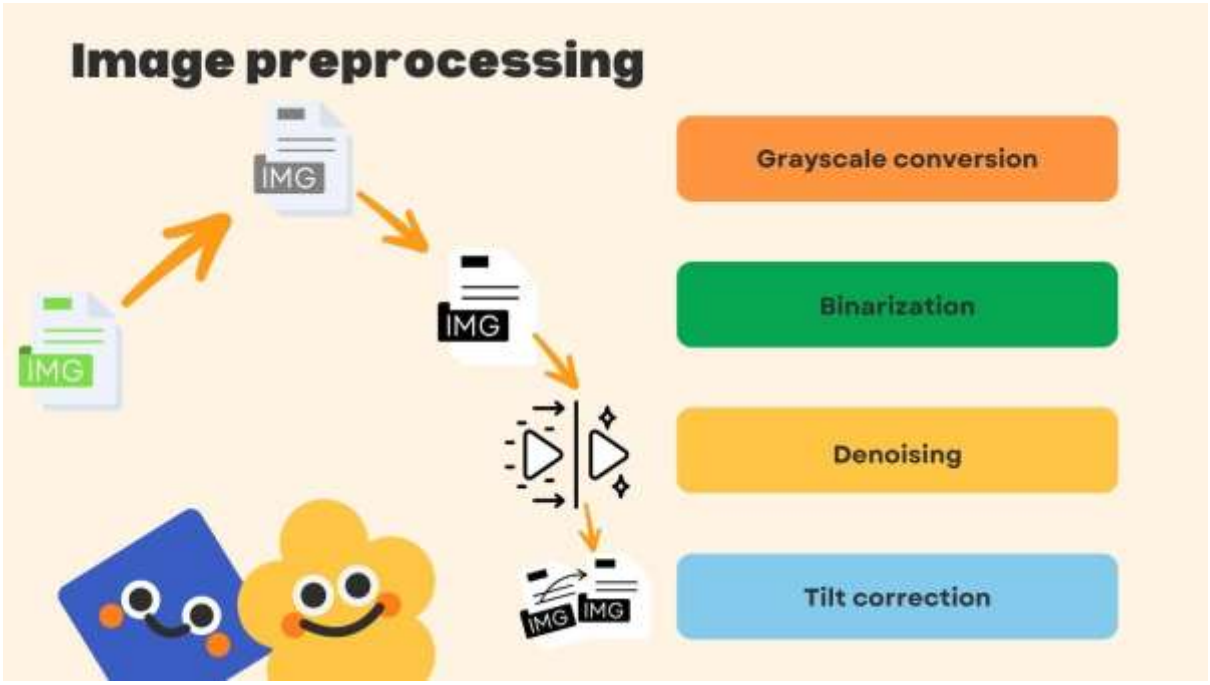
Constraint	**Solution**
Low-resolution images	Resize and enhance contrast
Noise	Median/Gaussian filtering
Varying illumination	CLAHE + adaptive thresholding
Different fonts	Proper segmentation and OCR
Efficient batch processing	Process images sequentially using OpenCV
Broken characters	Morphological closing

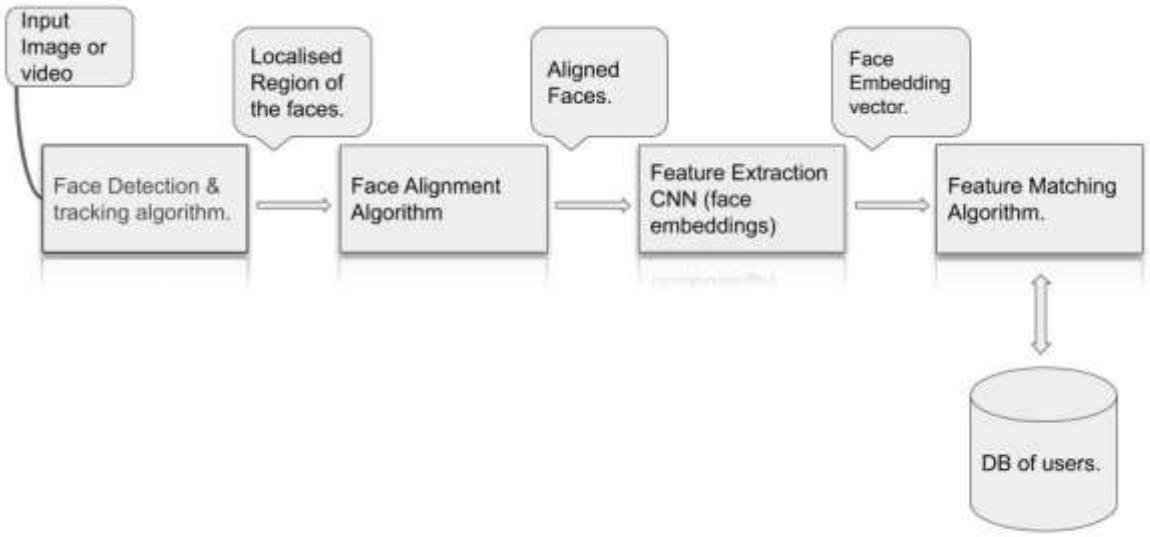
Result

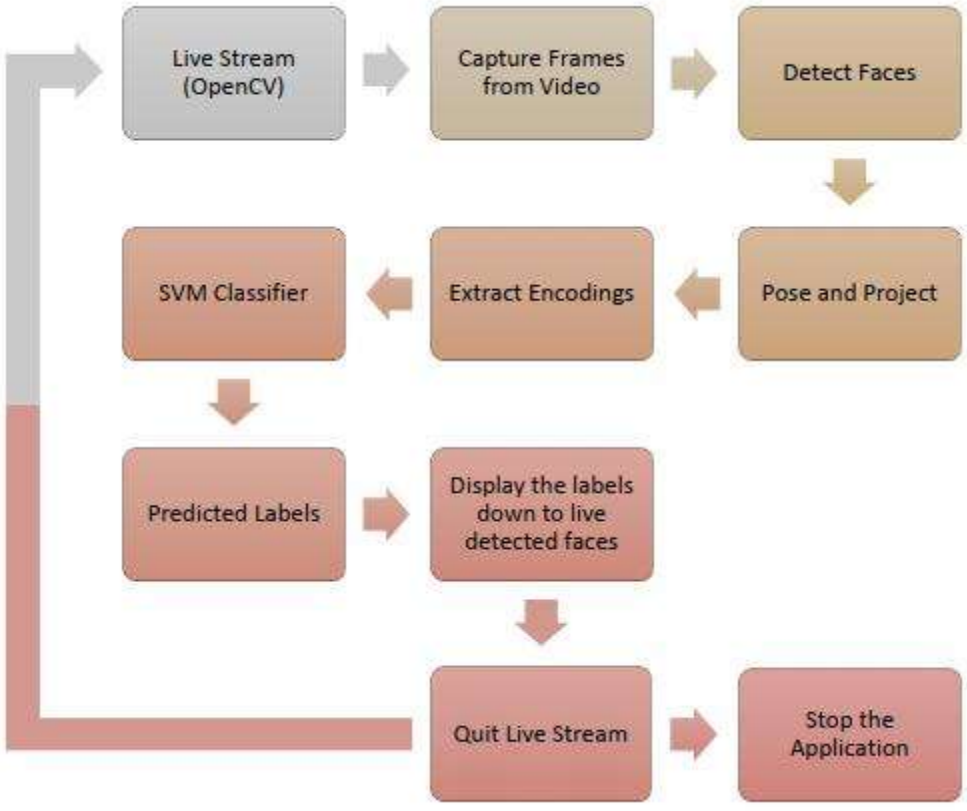
The proposed pipeline improves the quality of low-resolution scanned documents before OCR. Noise removal, contrast enhancement, adaptive thresholding, and segmentation improve text recognition while allowing multiple documents to be processed efficiently.

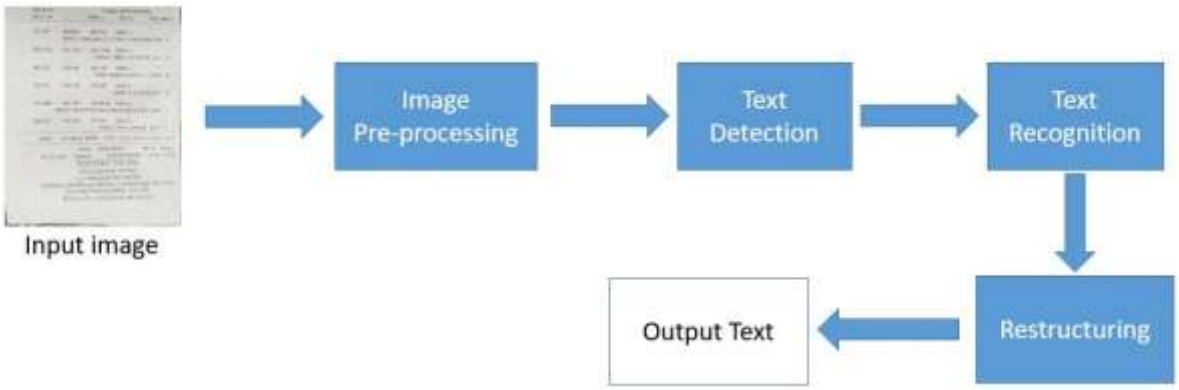
Overall OpenCV Pipeline






Input Image or video
Face Detection & tracking algorithm.
Localised Region of the faces.
Face Alignment Algorithm
Aligned Faces.
Feature Extraction CNN (face embeddings)
Face Embedding vector.
Feature Matching Algorithm.
DB of users.


Live Stream (OpenCV)
Capture Frames from Video
Detect Faces
Pose and Project
Extract Encodings
SVM Classifier
Predicted Labels
Display the labels down to live detected faces
Quit Live Stream
Stop the Application



Face Recognition:
Camera → Preprocessing → Face Detection → Feature Extraction → Recognition → Name

Video Surveillance:
CCTV → Preprocessing → Background Subtraction → Motion Detection → Activity Analysis → Alert

OCR:
Document → Grayscale → Denoising → Enhancement → Thresholding → Segmentation → OCR → Text

Code of Conduct:

I M JEEVITHA (192424257) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Jeevitha

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and	Identifies most constraints	Partial understanding; misses	Fails to identify constraints correctly

		identifies all relevant constraints accurately	with minor gaps	key constraints	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritizatio n	Limited analysis; weak prioritizatio n	No meaningful analysis or prioritizatio n
Solution Developme nt	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justificatio n	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided